PROOF OF WORK · TECHNICAL CASE STUDY

The model proposes, the server disposes.

AI-Native Developer | Full-Stack & AI Engineer
nex-store — a bilingual (FA/EN, RTL) AI-agent commerce platform, built solo over nine months. One receipt left the machine: a race-condition fix in Vercel's AI SDK, merged upstream.

100
registered tools · 98 domain + 2 meta, one defineTool() each · 100 .tool.ts files counted

2,507 / 195 / 169
unit+integration / e2e / eval scenarios
rg count · Playwright count · wc -l

167,304
TypeScript/TSX lines · ~94K prod / ~73K test · git ls-files + tokei cross-check

865 · 167
commits · active days · 2026-01-30 → 2026-11-07 · git log, single author

Receipts: issue #21548 → PR #21552 (merged, four of five commits co-authored, commit 47ba121) → @ai-sdk/langchain 3.0.119 + v6 backport 2.0.302 · 2026-09-28

CONTENTS

01 What this is
02 Architecture
03 The gate in the middle
04 Tools as contracts
05 Money safety, end to end
06 Untrusted output: PII and generated UI
07 The race condition
08 How this was built with AI
09 Testing, and the 48-hour answer
10 Trade-offs, on the record

> This section needs no trust, only a click.

Mahan Fakhimi
Tehran, Iran (UTC+3:30) · full overlap with EU hours · async-first · independent contractor (B2B)
dev.mahanfakhimi@gmail.com · Telegram @mahan_fk_dev · github.com/mahan-fk

2026-11-08

01 / What this is

The customer is a small-to-mid Iranian shop owner who sells through Telegram and can't hire a dev team. nex-store gives them a store that runs itself: the owner talks, the agent drafts a plan, the owner approves, the server executes. I built it alone over nine months. It is production-ready and pre-launch: no traffic, no users, and I don't claim otherwise. Every metric on the cover was computed from the repository, with the method stated next to it, and the one claim that leaves my machine is a click away.

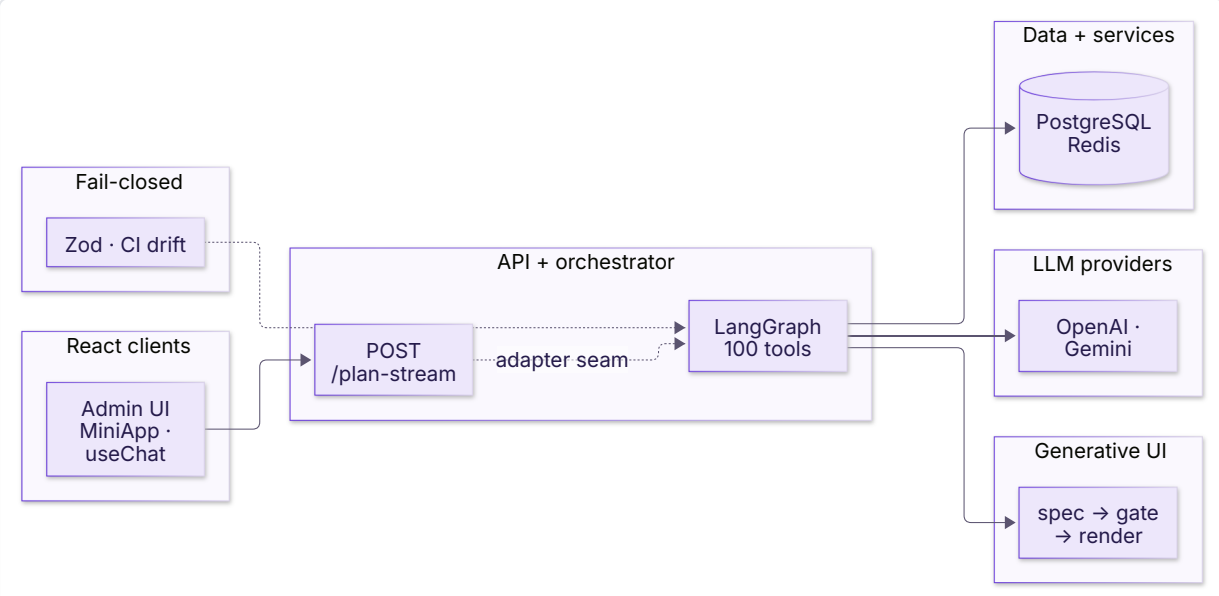
That one sentence drags in every hard constraint in the codebase. Persian and English share no script and no number conventions, and the agent writes to live business data, so amounts are attacker-controlled input. A hundred tools can't be governed by hand, so risk, caching and invalidation live in a contract registry enforced by CI. Owners watch plans execute on flaky mobile connections, so the streams are resumable and audited.

This document is a slice, not the whole system: the money path, the gates on untrusted output, tool governance, and one bug that outgrew my repo.

02 / Architecture

Three layers, clean division of labor. LangChain owns model I/O and tool definitions. LangGraph owns orchestration: a `StateGraph` with five nodes (intent → planner → approval → executor → replanner), compiled with a Redis checkpointer so an interrupted run resumes from state (`orchestrator.graph.ts:33-63`). Vercel AI SDK owns transport and UI; the `@ai-sdk/langchain` adapter is the seam between the two stacks, and §07 is a bug in that seam. The React 19 client renders typed message parts and 15 generative-UI primitives, and embeds in Telegram as a Mini App.

Tenancy is structural, not a filter: every key and bucket carries `shopId`, `wrapToolCall` refuses any call that loses tenant context, nothing crosses a boundary untyped, and every schema fails closed.



D1 — System architecture. Three layers, the adapter seam, and the generative-UI pipeline. Validation boundaries sit at every crossing. Traced from `orchestrator/graphs/orchestrator.graph.ts` and `ai-agent.controller.ts`.

03 / The gate in the middle

The graph is plan-and-execute with a human gate between planner and executor. Intent runs on a fast model behind a strict JSON schema and short-circuits chitchat to END.

The planner sees at most 8 retrieved tools out of 100 and emits typed steps, and a second structured call re-binds each step to a registry name, so a plan naming an unlisted tool fails at bind time. Sixteen state channels, three independent bounds on the loop.

The approval node is the part I'd defend first in any review:

- **Risk comes from contracts, not prose.** Every step is re-scored from the tool's registered declaration. A planner can't talk a declared high-write down, or a read tool up.
- **Fail-closed by default (CWE-754).** All trust and rate-limit signals start in the denying state, and a failed Redis check falls through to a human interrupt, never to auto-execution (`approval.node.ts:147-151`).
- **Graduated enforcement.** Read plans auto-execute. Low-writes auto-execute only with 5 or more consecutive approvals, opt-in, inside a 7-day window, under a 50-streak cap, and only when neither rate-limited (10 actions per 5 minutes) nor cooling down (30 minutes). Everything else hits `interrupt()`.
- **Trust updates are atomic.** Streak increment, threshold flip and decay run as one Redis Lua script, so two approvals landing together can't both read streak 4 and both advance it.

The executor never calls a tool directly: it prompts an agent bound to 8 core tools plus the 2 meta-tools, and everything else is dispatched by name with risk re-derived at call time. A hallucination loop re-prompts when the model claims success with zero tool calls, and a derived receipt makes "success" unrepresentable without a real call. The replanner owns stuck-detection with a hash over the last four steps, cycle detection, dedupe at 0.9 similarity, a stall counter that trips at three, and a hard cap of 12 iterations.

04 / Tools as contracts

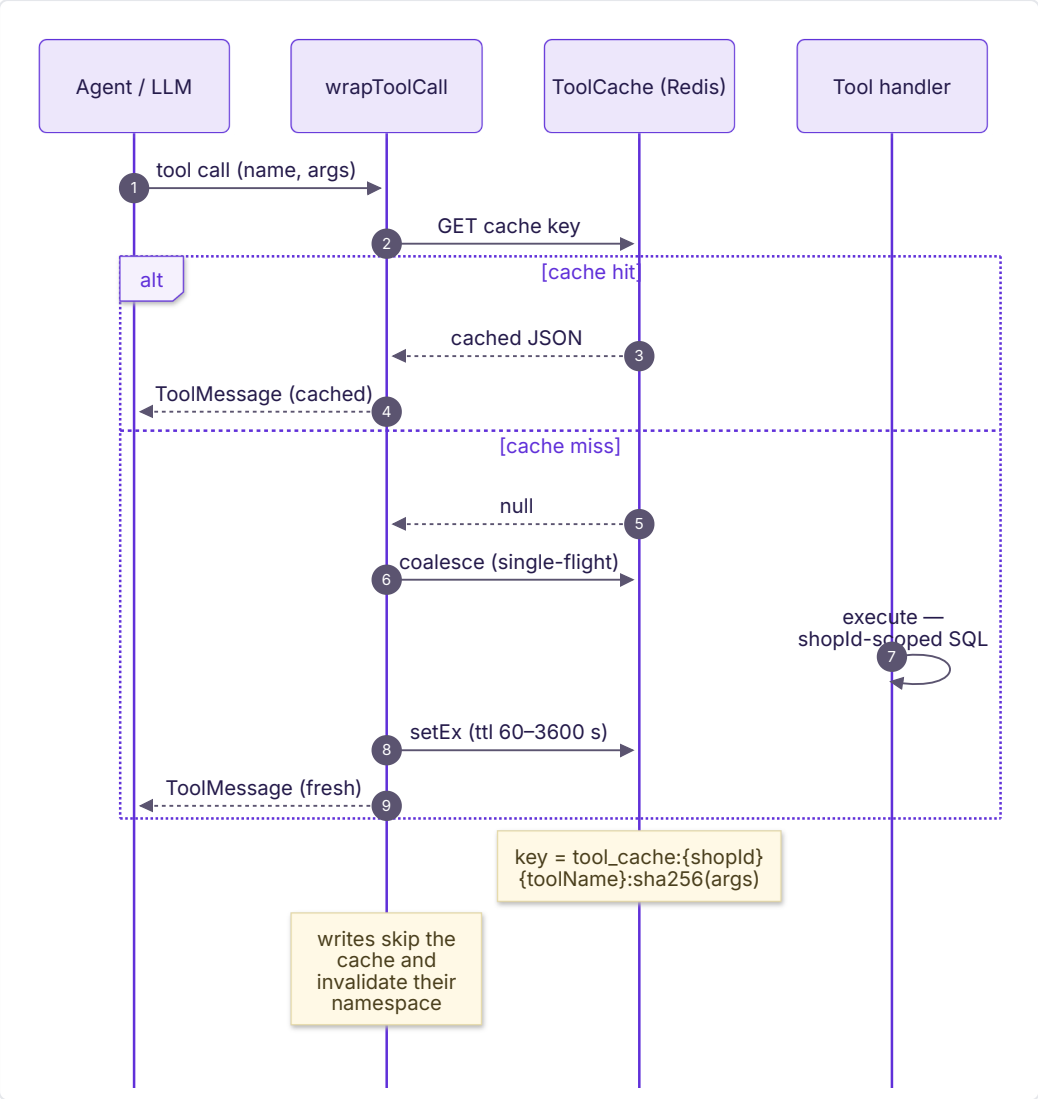
A tool isn't a function here. It's a registered contract: `defineTool()` requires `accessType`, `namespace`, `risk`, and either `sourceTable` or `mutatedTables`, plus a cache TTL and bilingual keywords. The registry forces Zod `.strict()` on every schema, because models emit near-miss keys (`max_price` for `maxPrice`) that default Zod silently strips. The tool would run with the filter missing, so strict parsing turns that into an error the model can correct. A CI job byte-compares the registry dump, so a renamed tool fails the build. Fourteen namespaces, snake_case, one naming spec with five assertions.

The cache is a tenant-isolation boundary. The key is the shop ID plus a SHA-256 over the tool name and canonical arguments:

apps/server/src/modules/ai-agent/tools/tool-cache.service.ts:41-47

```ts
export const buildToolCacheKey = (toolName: string, args: unknown, shopId: string) => {
  const hash = createHash("sha256")
    .update(`${toolName}:${canonicalJson(args)}`).digest("hex");
  return `${TOOL_CACHE_KEY_PREFIX}:${shopId}:${toolName}:${hash}`;
};
```

shopId is in every key, so tenants never share entries, and a spec pins that. Only reads are cached, TTLs are contract-declared (60 to 3,600 seconds), a Redis error fails open to execution, and writes invalidate their namespace. Bounded staleness buys one Redis round-trip instead of a query per repeated read; the worst case is a 3,600-second catalog read, mitigated by write-time invalidation.



D2 — The tool-call cache. Reads only; contract-declared TTLs (60–3,600 s); single-flight coalesce on miss; writes invalidate their namespace by SCAN+DEL. Traced from `tools/tool-cache.service.ts:41-47` and `agent-factory.service.ts:262-345`.

05 / Money safety, end to end

The threat model is boring on purpose. Price tampering: client-supplied amounts are attacker input (OWASP API3:2023). Excessive agency: a model that can set a discount can give the store away. The defense is architectural — the LLM never touches an amount, because no amount field exists in its attack surface.

Input first. Owners write سه هزار و پونصد تومان (se hazār o ponssad tomān, "3,500 toman"), or ۲٫۵ میلیون ریال, or "میلیون ۲ toman": colloquial, Persian-digit, mixed-script, all of it resolving to exact rial integers. The parser works in bigint fractions and never through JS `number`, for the reason the module records: IEEE-754 silently corrupts integers above 2^{53} (`money/rational.ts:1-13`). Where exactness is impossible, it refuses rather than rounds: ۱/۳ تومان is `UNSUPPORTED_SCALE`, a bare ۱۲۳ is `AMBIGUOUS_UNIT`. The model may extract a candidate amount. It never does arithmetic.

Then the write path. The agent's `orders_create_order` schema accepts exactly two fields, `userId` and `address` (`orders-create-order.tool.ts:12-25`). The charged total is recomputed from the catalog inside a transaction, under a row lock:

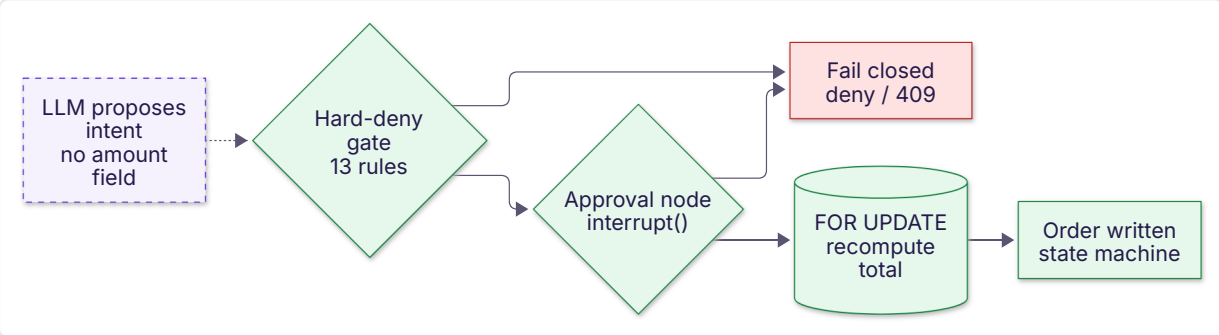
apps/server/src/modules/order/order.service.ts:226-247

```
const [product] = await tx.select().from(products)
  .where(and(eq(products.id, itemDto.productId),
            tenantScope(products.shopId, shopId)))
  .for("update");
// … inventory check …
const actualPrice = Number(product.price);
totalAmount += actualPrice * itemDto.quantity;
```

There is no client price to compare against. Complete mediation by construction: the model proposes the intent, and the server reads the real price itself. One honest caveat: `product.price` is a `decimal(12,2)` column Drizzle returns as a JS number, so above the parser the total is accumulated in floating point. The parsing layer is exact. The arithmetic over catalog prices is not.

> **13**
>
> hard-deny rules block or escalate financially dangerous calls at tool entry, regardless of approvals: discounts of 50% or more (detected even through base64, URL and JSON obfuscation, inside a 50 ms / 64 KB scan budget), bulk blasts to 100+ users, price swings of 50% or more, and every irreversible delete (`hard-deny-params.ts:309-424`). Of the 100 tools, 51 call `enforceHardDeny()` as the first statement of their handler.

Each item add re-enters `SELECT … FOR UPDATE` and writes the total as one SQL increment, so concurrent adds serialize on the row. Coupons derive their discount from the coupon row; the model supplies a code, never an amount. Finalization drops inventory with a conditional `UPDATE … WHERE inventory ⩾ quantity`; cash-on-delivery means paid at finalization. No agent tool can refund. And there is no idempotency key on the order path: a retried create opens a second order. Known gap, first in line for the gateway work.

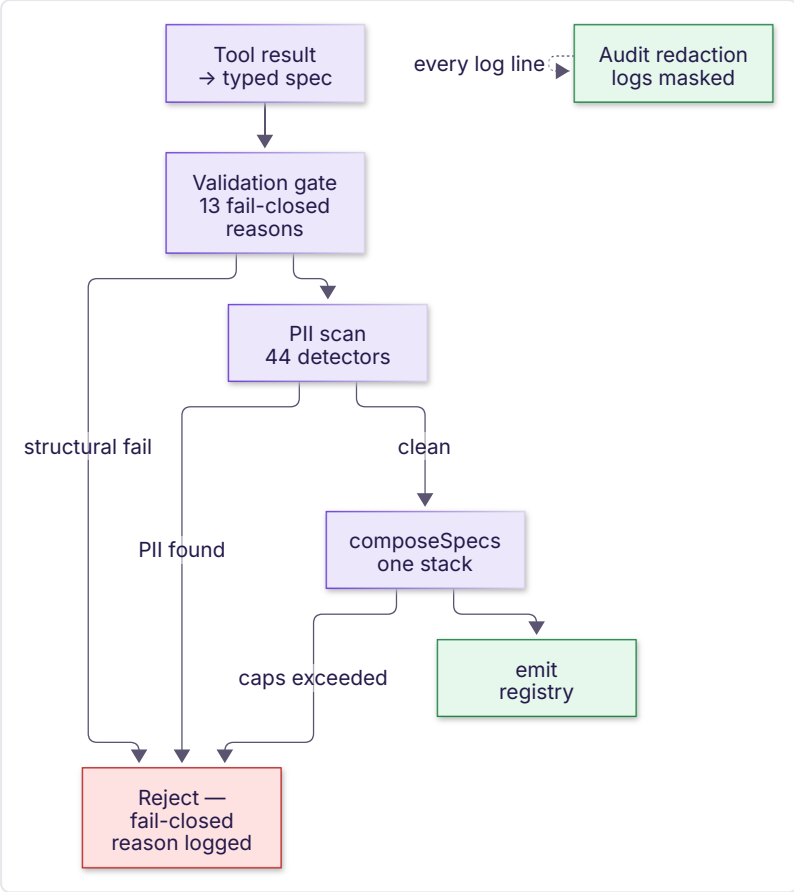


D3 — Payment guardrails. Violet dashed = untrusted model zone (intent only — no amount field exists). Green = server zone. Red = fail-closed. The charged total is always the server's own recomputation (`order.service.ts:226-247`, `hard-deny-params.ts:309-424`).

The bypass you'd expect is closed hardest: dynamic dispatch by name re-derives the target tool's declared risk and refuses when it outranks the current step, a `PRIVILEGE ESCALATION BLOCKED` path.

06 / Untrusted output: PII and generated UI

Two redaction layers, one engine. Layer one is declaration: every table gets a `FieldContract`, and five columns are declared PII outright, including `orders.address` and `shops.botToken`.



D4 — Untrusted-output pipeline. Tool result → typed spec → thirteen-reason gate → 44-detector PII scan → composed spec → client registry. A rejected spec is dropped, never half-rendered. Traced from `executor/generative-ui/validation-gate.ts:187-322`.

Layer two is value shape: 44 detectors across English, Persian and secrets catch the phone number pasted into an order note, or the JWT in a ticket body. A declared column is hidden without scanning a single value. A value scan catches the column that didn't exist when the contract was written. The engine fails closed: every regex compiles at module load, and a malformed pattern throws at boot, not in production.

When the executor turns a tool result into a UI spec, the validation gate rejects it for thirteen fail-closed reasons, each naming a failure a real user would otherwise meet: a frozen tab, a tenant leak, a privacy incident. Thirteen adversarial fixtures pin them, one reason per fixture:

FIXTURE — REJECTION REASON	FIXTURE — REJECTION REASON
`01-parse-fake-type` · schema parse failure	`08-forbidden-field-shop_id` · forbidden field (snake_case)
`02-parse-oversize-string` · byte budget on a field	`09-action-not-allowed` · action not in allow-list
`03-broken-references` · dangling reference	`10-action-params-invalid` action param schema invalid
`04-cycle-detected` · reference cycle	`11-pii-email` · PII — email
`05-depth-exceeded` · nesting depth	`12-pii-iban` · PII — IBAN
`06-byte-budget-exceeded` total spec byte budget	`13-pii-jwt` · PII — JWT
`07-forbidden-field-shopId` · forbidden field (camelCase)	Each fixture is one JSON file, pinned by a unit test.

The spec is built from the tool result, not written by the model. `buildGenerativeUISpec` parses the tool's JSON envelope and returns a typed spec, or nothing. The model contributes the tool call. Everything after it, element type to formatting, is deterministic code, so any spec defect is reproducible from a stored tool result. The shape is a closed union of 15 primitives, and the client registry is a `Record<SpecElementType, SpecElementComponent>`, so a sixteenth server primitive fails to

compile until a component exists for it. Actions are the smallest closed set in the system (retry, openInAdmin, markRead), because any action a spec can carry is an action an arbitrary row can trigger. The allow-list is a security boundary, not a UX choice.

07 / The race condition

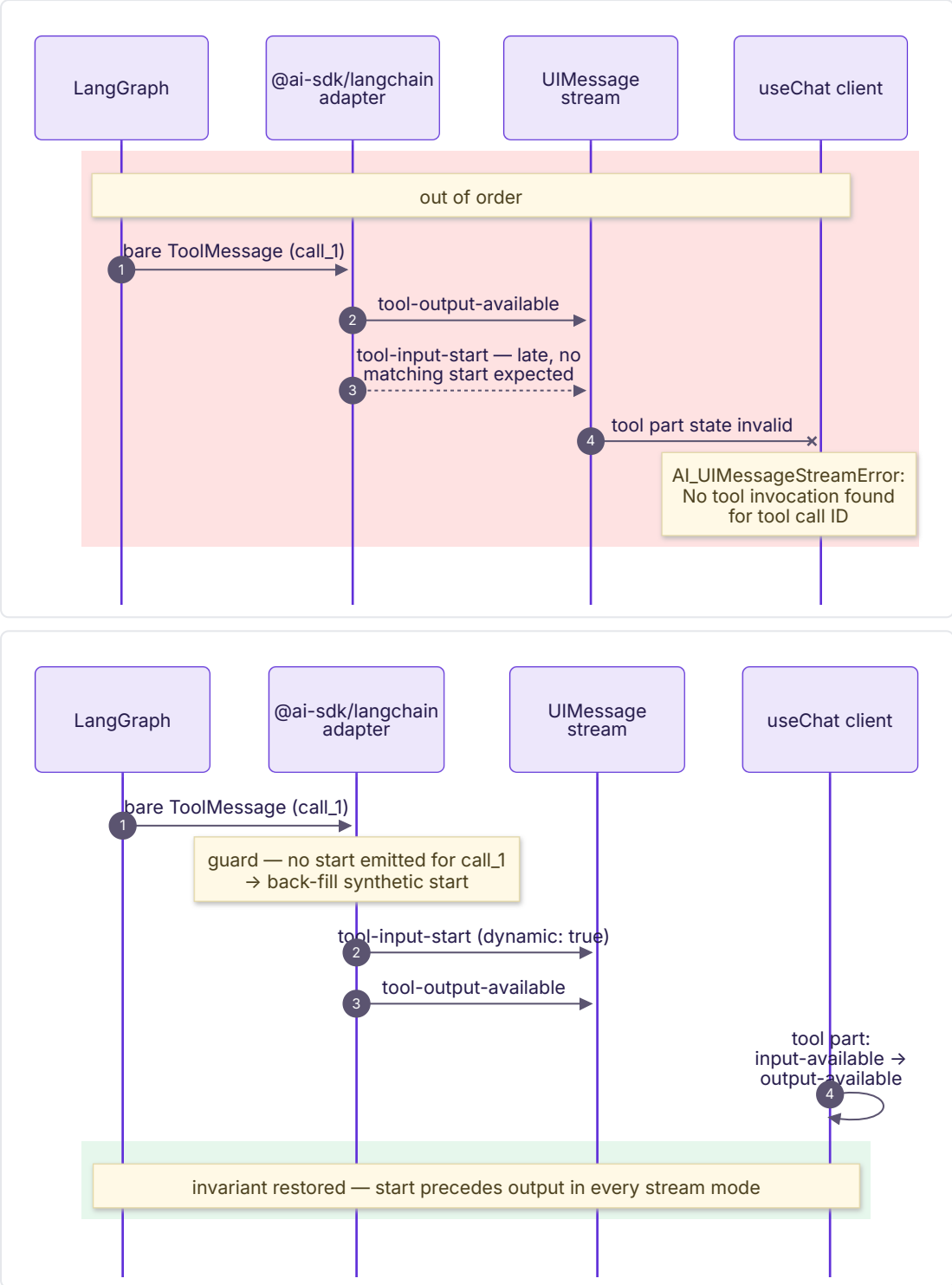
In July 2026, my chat UI started crashing mid-conversation. `AI_UIMessageStreamError: No tool invocation found for tool call ID.` Tool outputs were arriving in the UI message stream with no preceding `tool-input-start`, so the client's tool parts were being asked to complete a lifecycle that never began. Nothing failed at the source; the adapter emitted a complete-looking event stream that violated its own contract, which is what a confidently wrong answer looks like from the outside.

I shipped a defensive workaround first, ADR written before the fix: when a bare `ToolMessage` arrived, my transform layer emitted a synthetic `AIMessageChunk` with tool-call chunks in front of it (commit `fb6c8ccb`, 2026-07-25). No `node_modules` patching. The ADR carried removal steps.

Then I root-caused the adapter. The `@ai-sdk/langchain` event processor's completed-call guard was order-sensitive: in `messages` streaming mode, a bare LangGraph `ToolMessage` emitted `tool-output-available` with no `tool-input-start`. I filed issue #21548 on 2026-09-26 with a minimal reproduction, the observed-versus-expected chunk ordering, and a version matrix. Within hours, Vercel's agentic fix pipeline (`ai-sdk-factory[bot]`) reproduced it on `main` and opened PR #21552. The fix tracks unfinished tool calls per namespace and back-fills a synthetic `tool-input-start` when an output would otherwise arrive start-less. Four of its five commits carry `Co-authored-by: mahan-fk`. Maintainer `gr2m` approved and merged it on 2026-09-28, and it shipped that day as `@ai-sdk/langchain 3.0.119`, with a v6 backport (2.0.302).

The loop closed in my repo the same day. Commit `37038b1d` deletes the workaround and its tests, because the fixed adapter enforces the invariant natively. Report to release: about 48 hours. How often does a solo developer's bug report end up merged into an SDK whose core package sees roughly 45M downloads a week? I checked the numbers twice. And it wasn't luck: #21548 was the third adapter ordering bug I'd root-caused (#17863, #21146). The issue, the bot's reproduction, the co-authored commits, the release and the removal commit are all public. This section needs no trust, only a click.

> **The paper trail:** `fb6c8ccb` (workaround ADR, 2026-07-25) → issue #21548 filed (2026-09-26) → PR #21552 merged, commit `47ba121` (2026-09-28) → `@ai-sdk/langchain 3.0.119` + v6 backport 2.0.302 → `37038b1d`, workaround deleted (2026-09-28).



D5 — The race condition, before and after. Before (top): the output arrives with no preceding start, and the client's tool-part state machine rejects it — the crash. After (bottom): the adapter back-fills a synthetic `tool-input-start` (`dynamic: true`), so a start always precedes an output, in every stream mode. Reconstructed from issue #21548 and the merged PR #21552.

08 / How this was built with AI

I made every architectural decision. Coding agents accelerated implementation. I review every contract field and every invariant line by line, and read every failure personally; bulk output gets hunk-by-hunk review against the spec. The repo still carries that workflow in decision records, task specs, audit reports and ADRs.

Two columns, honestly:

WHAT THE MODEL DID WELL	WHAT I STEERED, FIXED, OR REJECTED
Scaffolding the 100-tool surface: factory boilerplate, Zod schemas, DTO wiring	The contract system itself: `defineTool()`, risk derivation, the CI drift-guard. Per-tool discipline doesn't survive agent-generated code unless it's enforced
Test drafts at volume (the 2,507 unit and integration cases started as agent drafts)	Every invariant test's assertion · the tenant-isolation cache test, the fail-closed trust defaults, the PII parity specs. They encode decisions, not boilerplate
Type migration drudgery across the monorepo	Rejecting the agent's first orchestrator designs that bypassed the approval node. Unbypassable by construction, not by prompt
Persian text normalization implementations	Reviewing against real Persian input (half-width digits, ZWNJ edge cases) — several drafts were wrong in ways only a Persian reader catches

The verification loop is the actual product: typecheck and scoped lint on every change, CI running the full suite plus a drift-guard and E2E smoke against an LLM stub, and a 169-scenario eval pool with a $3 nightly cap. Git records 9 AI co-authorship trailers. That number is low because I squash-review rather than rubber-stamp. Agents draft. The human decides. Typing was never the bottleneck.

09 / Testing, and the 48-hour answer

Unit specs prove logic with mocks. Integration specs prove the SQL against a real Postgres dialect (PGlite, from a prebuilt snapshot, so thousands of tests don't each pay the migration cost). E2E runs against a stubbed LLM across 11 areas, the only way an approval-flow test can assert determinism. The eval pool is stochastic, nightly, budget-capped, and its thresholds are published as gates (fa ≥ 70%, en ≥ 90%), not results, because a model-dependent stage doesn't get to grade itself.

And the 48-hour answer, as a runbook:

STAGE	AI DOES	HUMAN CHECKPOINT
Idea	Draft requirements from a problem	Pick the problem; define "done"
Architecture	Propose 2–3 designs with trade-offs	Choose; write the ADR
Contracts	Draft Zod schemas and types	Review every field · this is the attack surface
Implementation	Scaffolding, boilerplate, bulk edits	Review diffs hunk-by-hunk vs the spec
Tests	Draft cases, fixtures	Own the invariants; verify assertions lie to no one
Checks	Run typecheck, lint, suites; triage failures	Read failures personally; no skipped tests
Deploy / monitor	Wire CI, dashboards, log redaction	Decide rollout gates; own incidents

10 / Trade-offs, on the record

ADR

TTL cache-aside with contract-declared TTLs and write-time namespace invalidation.

Rejected: event-driven invalidation only — plumbing for 100 tools and every mutation path. **Accepted cost:** worst-case 3,600 s staleness, mitigated by write invalidation; SCAN is O(keyspace-page), fine pre-launch.

ADR

Fail-closed everywhere a safety signal can fail.

Rejected: fail-open — better availability, invisible regressions. **Accepted cost:** a Redis outage means approvals instead of auto-execution; a detector bug means a rejected spec, not a leaked IBAN. An owner tapping "approve" beats a silent bypass.

ADR

In-code adapter workaround over patch-package, removed on upstream fix.

Rejected: forking the SDK (breaks on every upgrade) and waiting (the UI was crashing). **Accepted cost:** nine weeks carrying a documented workaround with a pre-written removal procedure, validated when the removal commit landed with the fix.

Patterns worth a line each, with the file that proves them:

PATTERN — CITATION		PATTERN — CITATION	
Stream audit tee · audit rides alongside delivery, never blocking the client	`stream-tee-audit.ts`	Context budgets · history sanitized, tokens counted, calls budgeted	`token-counter.ts`
Per-subsystem metrics · agent, tool, cache, stream as services	`observability/services/*`	Schema sanitization · schemas cleaned before model binding	`schema-sanitizer.ts`
PII-safe logging · errors scrubbed, payloads safe-stringified	`scrub-error.ts`	Risk-visible UI · per-step risk badge + approval panel	`StepRiskBadge.tsx`
Prototype-pollution guard param merging rejects __proto__ keys	`pollution-guard.ts`	Contract drift tests · naming, catalog, schema parity pinned	`tool-naming.unit.spec.ts`
Token accounting · usage counted per execution	`token-usage.ts`	Audit at persistence · audit log + thread ownership as entities	`audit-log.schema.ts`
Multi-provider interop · Gemini thought-signatures keep reasoning models in the loop	`gemini-thought-signature.ts`	Bilingual normalization · Persian normalization as reusable utilities	`normalize-fa.ts`
Prompts as data · versioned per-language JSON, composed at runtime	`prompts/composer.ts`	Dynamic dispatch meta-tool the trust gate blocks risk escalation	`call-tool-by-name.tool.ts`

What I'd change: cut scope earlier. The contract system arrived after the tool count passed about 40, and the `defineTool()` retrofit was the least fun week of the project; contract-first would have been cheaper. The admin UI's non-chat pages sit at roughly 85% by my own tracking, and context switches during the March–May gap cost me a third of the calendar. The June–October stretch is where the platform actually got built.

Next: gateway-integrated payments, with idempotency at that boundary. Then launch. The role page says AI can write the code, and you still own the engineering. That's how this repo was run.

Mahan Fakhimi — full-stack and AI engineer, six years of paid work across international freelancing, Iranian product companies, and a Canadian software company. React/TypeScript, Node/NestJS, LangChain/LangGraph, Vercel AI SDK. Tehran, Iran (UTC+3:30), full overlap with European business hours. Async-first; written English is the strongest channel (C1 written, B2 spoken), Persian native. Independent contractor (B2B). The repo is private; interview read access on request.
dev.mahanfakhimi@gmail.com · Telegram @mahan_fk_dev · github.com/mahan-fk